

Assignment 02 - FSStat

Problem Analysis

The assignment requires an asynchronous library method `getFSReport(D, MaxFS, NB)` that recursively scans a directory tree and computes the total number of regular files and a distribution over $NB + 1$ file-size bands.

The first NB bands partition the closed interval $[0, \text{MaxFS}]$. The last band counts only files whose size is strictly greater than MaxFS . A file whose size is exactly MaxFS belongs to the last ordinary band.

The main concurrent aspects are directory traversal, file metadata access, result aggregation, and error isolation. The implementation uses an immutable `FSReport` record. Single-file reports are merged functionally, avoiding shared mutable counters and synchronization.

Adopted Strategy

Event-Loop / Asynchronous Version

`pcd.ass02.eventloop.EventLoopFSStatLib` exposes `CompletableFuture<FSReport>`. Directory listing and file size operations are submitted to an executor, and composition is expressed with `thenCompose`, `allOf`, and `thenApply`. The caller receives a future immediately.

Reactive Version

`pcd.ass02.reactive.ReactiveFSStatLib` exposes `Single<FSReport>` and uses RxJava Flowable streams. Paths are streamed recursively, regular files are mapped to one-file reports, and the final report is produced with `reduce`.

The implementation uses a custom Rx Scheduler backed by a `ScheduledExecutorService`, so scheduler lifecycle is explicit and no global RxJava scheduler thread is leaked when running through Maven.

Virtual-Thread Version

`pcd.ass02.virtualthreads.VirtualThreadFSStatLib` exposes `CompletableFuture<FSReport>`. Recursive subdirectories are submitted to a virtual-thread-

per-task executor. The code uses direct blocking waits inside virtual threads, which keeps the structure simple without occupying platform threads.

Example Program

```
.\mvnw.cmd exec:java "-Dexec.args=eventloop . 1048576 8"  
.\mvnw.cmd exec:java "-Dexec.args=reactive . 1048576 8"  
.\mvnw.cmd exec:java "-Dexec.args=vt . 1048576 8"
```

Benchmark

`pcd.ass02.benchmark.FSStatBenchmark` creates a temporary deterministic dataset with 1200 files distributed across nested directories. It executes one warm-up run and then 5 measured runs for each implementation. All three reports are compared for equality.

```
.\mvnw.cmd exec:java "-Dexec.mainClass=pcd.ass02.benchmark.FSStatBenchmark"
```

Version	Files	Over MaxFS	Avg ms	Min ms	Bands
eventloop	1200	599	67.93	54.63	[146, 5, 145, 5, 145, 5, 145, 5, 599]
reactive	1200	599	116.04	100.96	[146, 5, 145, 5, 145, 5, 145, 5, 599]
virtual threads	1200	599	55.12	47.53	[146, 5, 145, 5, 145, 5, 145, 5, 599]

The benchmark is intentionally small and mostly validates behaviour and overhead. The virtual-thread version is fastest in this run. The reactive version has higher overhead on this dataset due to stream and scheduler machinery.

Testing

JUnit tests cover band mapping, immutable merge semantics, and cross-version equivalence on a generated temporary directory tree.

```
.\mvnw.cmd test
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Packaging was also verified with `.\mvnw.cmd package`. The shaded jar is generated successfully; Maven Shade reports only the expected manifest overlap warning from bundled dependencies.